Cycle-Time Optimization of Reconfigurable Robotic Pick-and-Place Cells in a Digital Twin

Abstract—Layout design for robotic pick-and-place cells is dominated by the cost of evaluating candidate configurations: each evaluation conventionally requires a sampling-based motion plan in a high-fidelity simulator, which is both slow and stochastic. We present a declarative reconfigurable-cell framework (ROS 2 Jazzy + Gazebo Harmonic + MoveIt 2, UR5 + RG2 gripper) in which an optimizer searches over station poses and visit order using a deterministic joint-travel surrogate as its objective, rather than the simulator. The surrogate evaluates a configuration by chaining canonical, elbow-up inverse-kinematics solutions and summing joint-space travel, achieving $< 0.0003\%$ run-to-run spread. Our central contribution is an honest validation of this surrogate against real planned motion time. A naive comparison against a randomized feasibility planner (RRTConnect) yields no reliable correlation—the Pearson coefficient flips sign between repetitions ($+0.057$ vs. -0.550)—because per-plan path-length variance ($\sim 96\%$) swamps the between-configuration signal. Once the measurement is aligned with what the surrogate models —an optimizing planner (RRT-star), kinematic continuity (joint-to-joint goals), and a fixed visit order—the surrogate strongly predicts converged-optimal motion time: Pearson $r = 0.880$, Spearman $\rho = 0.943$ ($n = 6$). Optimizing against the surrogate with simulated annealing beats an equal-budget random baseline 5/5 (mean -17.4%), and the predicted reduction transfers to real motion: pooled $r = 0.876$, with optimized configurations running 19.4% faster in real RRTstar motion. Finally, an optimized configuration executes a full pick–place–pick–place relay end-to-end in the high-fidelity twin (a warmup and one measured trial, both succeeding with no planning failures). A fair comparison of five black-box optimizers (Bayesian optimization, CMA-ES, simulated annealing, genetic algorithm, particle swarm) on the same surrogate shows no single winner—the top three tie within noise—while simulated annealing is the most cost-efficient and reliable, and random placement finds no valid layout. Adding the robot base pose to the search reveals a structural property: the surrogate and its constraints are base-relative, so the absolute mount pose is cost-neutral. Finally, the approach generalizes: across 50 seeds and two robot arms (UR5 and UR10), simulated annealing beats an equal-budget random baseline (46/50 and 45/50), and the surrogate's correlation with real motion transfers to the UR10 ($r = 0.83$, bootstrap 95% CI excluding zero). We also report the boundary of the result: the most aggressive minimal-travel layouts are infeasible for the twin's default randomized planner, so time prediction and full-twin executability are distinct guarantees.

Index Terms—robotic cell design, motion planning, digital twin, simulated annealing, surrogate optimization, reconfigurable manufacturing.

I. Introduction

The throughput of a robotic pick-and-place cell is largely fixed at design time by the relative placement of its work stations and the order in which the robot visits them. Searching this layout space is attractive but expensive: the natural objective—real cycle time—is obtained only by executing or planning each candidate in a simulator. Sampling-based planners such as RRTConnect [1] are fast but stochastic, so a single configuration must be evaluated many times to average out path-length variance, and a global search over hundreds of candidates becomes intractable.

A common remedy is a cheap surrogate objective. The risk is that the surrogate optimizes a proxy that does not track the quantity of interest. This paper takes the validation of that proxy as seriously as the optimization itself. We build a declarative, reconfigurable cell on ROS 2 and Gazebo (Section III), define a deterministic joint-travel surrogate (Section IV), and then ask—repeatedly and honestly—whether it predicts real planned motion time (Section V). We deliberately report two failed attempts before the successful one, because the failure mode is instructive: the surrogate is only predictive when the measurement is methodologically aligned with what it models. We then optimize against the validated surrogate, confirm the predicted gains transfer to real planned motion (Section V), and ask which optimizer to use (Section VI). We extend the search to the robot base pose and report a structural finding on its cost-neutrality (Section VII), establish that the approach generalizes across 50 seeds and two robot arms (Section VIII), and execute an optimized configuration end-to-end in the high-fidelity twin (Section IX).

Contributions. (i) A declarative reconfigurable-cell framework with a reusable headless IK oracle shared between synthesis and simulation. (ii) A deterministic joint-travel surrogate with $< 0.0003\%$ spread, suitable as a stable optimization objective. (iii) A rigorous surrogate-to-real validation establishing that the surrogate predicts an optimizing planner's motion time ($r = 0.88$), together with a clear account of why it fails against a randomized planner. (iv) An end-to-end demonstration that simulated-annealing optimization against the surrogate yields configurations that are faster in real motion and executable in the twin, and an honest characterization of where executability breaks down. (v) A fair comparison of five black-box optimizers on the identical surrogate, oracle, and budget, showing that no method dominates and that simulated annealing is justified by cost and reliability rather than peak quality. (vi) A full-cell synthesis formulation that adds the robot base pose to the decision space, together with a structural finding that the absolute

base pose is cost-neutral under a cycle-time objective. (vii) Generalization evidence across 50 seeds and two robot arms (UR5 and UR10) with bootstrapped confidence intervals, demonstrating that the surrogate-optimization approach transfers.

II. Related Work

Robotic cell layout optimization. Optimizing the physical layout of a robotic cell is a long-standing problem, and metaheuristics are a common tool. Sánchez-Sosa and Chavero-Navarrete [11] use a genetic algorithm to minimize cell area and inter-station transfer distance under reach and interference constraints, validating the result against a physical screwdriving cell, while a classical commercial tool encodes the same objective with a modified simulated annealer [12]. More recent work couples the layout search to a digital twin: Wang et al. [13] apply multi-agent cooperative swarm learning to dynamically re-optimize reconfigurable assembly-cell layouts inside a twin. All of these evaluate a candidate configuration by executing the digital twin or a simulator directly. We instead decouple the optimizer from the high-fidelity simulator through a validated kinematic proxy, so the search itself never invokes a planner.

Surrogate-assisted layout and motion optimization. Closest to our work is surrogate-assisted optimization, a well-established idea in evolutionary computation [21]. Kawabe et al. [10] jointly optimize motion planning and cell layout using a trained surrogate that estimates a manipulator's operation time with 98% accuracy at roughly 1/400 of the computation of full planning, and a related hierarchical framework couples an adaptive genetic algorithm with a surrogate-assisted RRT for multi-target placement [14]. Both rely on a learned surrogate trained on planner outputs. Our surrogate is instead a closed-form kinematic formula—a sum of canonical joint-space travel—that needs no training data and is deterministic by construction. We additionally characterize the conditions under which a kinematic surrogate does and does not predict real planner output (Section V), a methodological finding neither work reports.

Sampling-based motion planning. Our real-motion measurements use sampling-based planners. Beyond RRT-Connect [1] and the asymptotically optimal RRTstar [2], the probabilistic-roadmap method [15] established the paradigm for high-dimensional configuration spaces. The per-plan path-length variance we document in Section V—precisely why a single twin evaluation is an unreliable objective—is a known property of this algorithm family [16].

Digital-twin-assisted cell design. Finally, digital twins are increasingly used in robotic cell design [20]. Saavedra Sueldo et al. [17] present a ROS-based architecture for rapid twin development of robotized manufacturing systems, and Kousi et al. [18] use a twin to design and reconfigure human–robot collaborative assembly lines.

Fig. 1. The high-fidelity Gazebo Harmonic twin for an optimized configuration (val_0): a UR5 + RG2 arm on a pedestal serves three conveyor stations against a warehouse backdrop. Station poses and visit order are produced by the optimizer; the green cube is the transferred part.

Those twins primarily support runtime reconfiguration. Ours is a design-time twin whose headless IK oracle is shared, by import, between the optimizer and the simulator, so a synthesized configuration is valid in the twin by construction.

III. System Architecture

The platform is a digital twin built on ROS 2 Jazzy and Gazebo Harmonic [22], with a Universal Robots UR5 arm and an OnRobot RG2 two-finger gripper [25] (Fig. 1). Motion planning uses MoveIt 2 with the OMPL planner suite. A cell is described declaratively in a locked schema—robot mount pose, belt geometry, part geometry, and a list of conveyor stations (each with an (x, y, ψ) pose), plus a relay task as an ordered list of pick/place operations. A generator realizes a configuration document into a scene.yaml and task.yaml consumed by the simulation.

Two components are shared, by import, between the headless optimizer and the live simulator, which is what makes the surrogate trustworthy: (1) an IK reachability guard that rejects any configuration with an unreachable station, and (2) the inverse-kinematics oracle used to score configurations. Grasping in the twin is implemented as a welded detachable joint, so the focus of this study is reach and transit motion rather than contact dynamics.

A practical but important lesson during twin construction: the ros_gz_sim spawn utility silently ignores a pose embedded in an SDF string, so every runtime fixture must be spawned via explicit -x/-y/-z/-Y flags; otherwise all fixtures stack at the world origin. Belt-support slabs are added so the part rests at a stable grasp height.

IV. Deterministic Joint-Travel Surrogate

The surrogate estimates the motion cost of a configuration without invoking a planner. For each target pose in the (ordered) relay, it computes inverse kinematics and selects the canonical solution—the collision-free branch closest to a fixed elbow-up seed over a deterministic seed bank—then sums the joint-space distance traveled between consecutive canonical configurations. Selecting a canonical branch is essential: a generic IK call random-restarts and jumps between elbow-up/elbow-down solutions, which made an early version of the surrogate vary by 50–127% on identical inputs. With canonical selection the surrogate is effectively deterministic, with measured run-to-run spread below 0.0003%. This stability is a prerequisite for using it as an optimization objective: simulated annealing requires that two evaluations of the same point return the same value.

The visit order is fixed separately by a brute-force traveling-salesman solution [19] over the stations, so that the surrogate and any downstream real measurement use an identical order.

V. Surrogate-to-Real Validation

We treat "does the surrogate predict real motion time?" as a hard gate that must pass before any optimization result is trusted. The validation set val_hb contains six valid configurations spanning a range of surrogate values, all sharing one robot mount.

A. Two honest negatives

Attempt 1 (inconclusive). Measuring full cycle time in the twin mixed in $\sim 130\,\mathrm{s}$ of configuration-independent reset/settle overhead and suffered $\sim 2\times$ run-to-run motion variance from the randomized planner; no clean trend emerged, and sim instability prevented collecting enough runs to average it out.

Attempt 2 (reliable negative). A headless probe stripped the fixed overhead and the wall-clock noise by summing planned trajectory duration with the real RRTConnect planner, averaged best-of-10. The result was explicitly not a relationship: the Pearson coefficient flipped sign between repetitions of the same experiment ($+0.057$ at 12 plans vs. -0.550 at 40 plans). The cause is that RRTConnect path duration retains $\sim 96\%$ per-plan spread even at best-of-10, and frequently fails on the collision-constrained reaches. A coefficient that flips sign is measuring noise; we report it as such rather than selecting the favorable run.

B. Aligning the measurement (Attempt 3)

The diagnosis was that minimal joint travel is the wrong target for a randomized, feasibility-only sampler. Three changes align the real measurement with the quantity the surrogate models:

1) Optimizing planner. RRTConnect → RRTstar [2], which is asymptotically optimal in path cost. This required injecting a group-qualified planner configuration so RRTstar is selectable for the arm group.
2) Kinematic continuity. The relay is planned joint-to-joint: each station's goal is the collision-free IK branch closest to the previous station's joint state, eliminating spurious elbow-flip excursions. A transfer $A \to B$ excludes both endpoint belts (leaving A and approaching B are required, not collisions), while every other belt remains an obstacle.
3) Fixed visit order. The brute-force TSP order is shared by surrogate and real measurement.

With continuity, five of six configurations become near-deterministic (per-plan spread 89% → 0%); the lone exception, val_2, is RRTstar occasionally failing to converge to its optimum.

TABLE I
Surrogate vs. real RRTstar motion time on val_hb ($n = 6$, RRTstar $6\,\mathrm{s} \times 10$ plans). Min is the converged-optimal estimate.

config	surrogate	min (s)	mean (s)	median (s)	std
val_0	5.17	6.81	6.81	6.81	0.00
val_1	6.38	8.06	9.14	8.92	0.98
val_2	6.60	7.35	12.58	10.40	3.79
val_4	6.92	8.56	8.61	8.61	0.02
val_3	6.96	9.03	9.08	9.10	0.03
val_5	7.36	9.11	9.52	9.11	1.22

TABLE II
Correlation of the surrogate with real motion time, by estimator.

estimator	Pearson r	Spearman ρ
min (converged-optimal)	0.880	0.943
median	0.704	0.543
mean	0.486	0.371

C. Result

Table I reports the surrogate and the real RRTstar motion time (minimum, mean, median, std over 10 plans). Because RRTstar is asymptotically optimal, the minimum planned duration is the best estimate of its converged-optimal path cost; the mean is contaminated by non-converged runs (visibly val_2, whose min $7.35\,\mathrm{s}$ matches its mid-range surrogate while unlucky runs inflate its mean to $12.6\,\mathrm{s}$). Table II compares correlation under each estimator; the min is both the most principled and the strongest. Fig. 2 shows the scatter and fit.

Once the measurement matches what the surrogate models, the deterministic joint-travel surrogate strongly predicts the optimizing planner's motion time, and the rank order is almost exact ($\rho = 0.943$). Attempt 2's negative is therefore a property of the randomized planner/sampler, not of the surrogate. The gate passes, justifying optimization against the surrogate.

D. Optimizing the surrogate reduces real motion

A final validation step confirms that optimizing the surrogate lowers real motion, not merely that the two

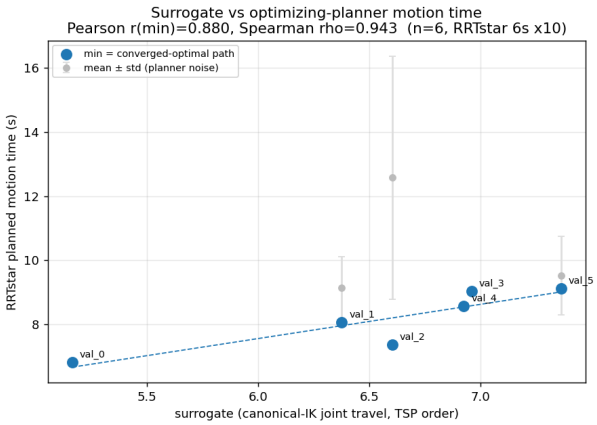

Fig. 2. Surrogate vs. real RRTstar motion time on val_hb. Filled points and fit line use the converged-optimal (min) estimate ($r = 0.880$, $\rho = 0.943$); faded points show mean ± std for transparency.

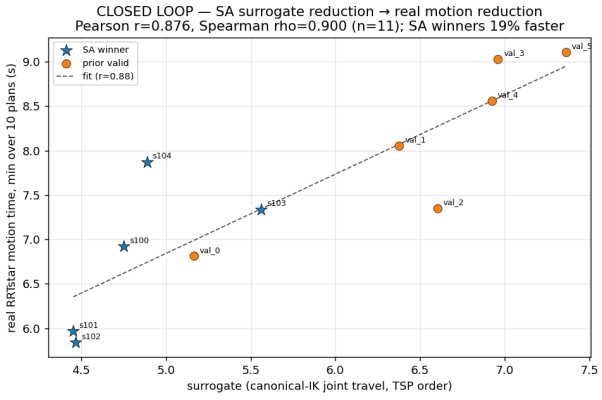

Fig. 3. Pooled surrogate vs. real motion ($n = 11$). Optimized winners (low end) and random valids lie on the same trend; the surrogate reduction maps onto a real-motion reduction.

TABLE III
Optimizing the surrogate transfers to real motion: simulated-annealing winners vs. random valids (pooled $n = 11$, min motion).

group	mean real motion (s)	pooled correlation
SA winners ($n = 5$)	6.79	$r = 0.876$, $\rho = 0.900$
random valids ($n = 5$)	8.42	
relative improvement: −19.4% real motion (−17.4% surrogate)		

correlate. Optimizing station poses and the visit order against the surrogate with simulated annealing (the optimizer itself is examined in Section VI) yields five winners; pooled with the six gate configurations ($n = 11$), the surrogate–real correlation holds on min motion (Pearson $r = 0.876$, Spearman $\rho = 0.900$). All five winners are deterministic over ten plans and sit at the low real-motion end, averaging 6.79 s versus 8.42 s for random valid configurations—a 19.4% real-motion reduction that matches the 17.4% surrogate gain of the same five-seed run (Table III, Fig. 3). Optimizing the cheap proxy produces a real optimizing-planner speedup of the same magnitude.

VI. Optimizer Comparison

Optimization uses simulated annealing [3] over station poses with a brute-force TSP visit order; on an initial five-seed run it beat an equal-budget random baseline 5/5 (−17.4%, above), and Section VIII scales this to fifty seeds. The optimizer sits inside the synthesis-and-validation pipeline of Fig. 4. But is simulated annealing the right optimizer? We answer with a controlled comparison of five established black-box optimizers—Bayesian optimization with a Gaussian process [7], CMA-ES [8], simulated annealing, and a genetic algorithm [24] and particle-swarm optimization [23] (both via pymoo [9])—all minimizing the same deterministic surrogate, over the same decision space (per-station placement in the reach annulus, visit order fixed by the same TSP), behind the same validity oracle, under an identical 140-evaluation budget, across the same six seeds. Infeasible candidates receive an identical penalty, so any difference is attributable to the optimizer, not the setup.

We report the outcome honestly: no optimizer dominates. The top three—Bayesian optimization (5.74±0.71), CMA-ES (5.80 ± 0.65), and simulated annealing (5.91 ± 1.03)—are a statistical tie, separated by far less than the seed-to-seed standard deviation (Table IV, Fig. 5). Simulated annealing is therefore not uniquely best in solution quality; its justification is cost and reliability. It reaches the tied quality at 4.7 s per seed with 6/6 feasibility, whereas Bayesian optimization matches that quality only at 121 s per seed (its Gaussian-process refits dominate the runtime), and the genetic algorithm fails to find any valid layout on one of six seeds. Critically, uniform random placement over the same reach annulus finds zero valid three-station layouts within the budget on every seed—a well-separated, reachable, non-overlapping triple is a roughly one-percent-density target—so optimization is necessary, and the gap between any optimizer and no optimizer dwarfs the gaps between optimizers. A secondary check validates the visit-order treatment: letting simulated annealing search the order jointly, rather than fixing it by TSP, reaches a marginally lower cost (≈5.05) but only by spending ∼100× the evaluation budget and does not change the ranking—because the surrogate is the joint travel the TSP minimizes, the TSP order is already optimal for any placement.

VII. Full-Cell Synthesis and Base-Pose Finding

The optimizer so far places stations around a fixed robot mount. We next add the robot base pose (x, y, ψ) to the decision space—sampling stations in the reach annulus relative to the sampled base and making the generator base-yaw aware—so the optimizer chooses where to mount the robot and where to place the stations. A sanity run that freezes the base reproduces the fixed-mount result

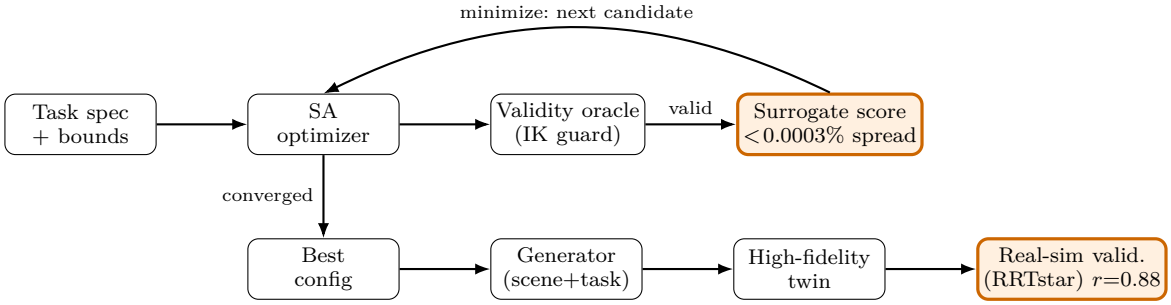

Fig. 4. The synthesis-and-validation pipeline. Simulated annealing proposes station/base placements; the shared headless IK oracle rejects invalid candidates; the deterministic surrogate scores the rest (<0.0003% run-to-run spread); the loop minimizes the surrogate. The best configuration is realized by the generator into the high-fidelity twin and independently validated against the real RRTstar planner ($r = 0.88$ on the curated gate; simulated annealing beats random 46/50 at scale, Section VIII).

TABLE IV
Five black-box optimizers on the same surrogate (6 seeds, equal 140-evaluation budget, three stations). No method dominates; the top three tie within the seed-to-seed std.

optimizer	cost (mean±std)	best	feasible	wall/seed
Bayesian opt. (GP)	5.74 ± 0.71	4.90	6/6	121 s
CMA-ES	5.80 ± 0.65	5.29	6/6	3.2 s
simulated annealing	5.91 ± 1.03	5.01	6/6	4.7 s
genetic algorithm	6.05 ± 0.98	5.42	5/6	2.0 s
particle swarm	6.24 ± 0.84	5.30	6/6	2.0 s
random placement	—	—	0/6	0.1 s

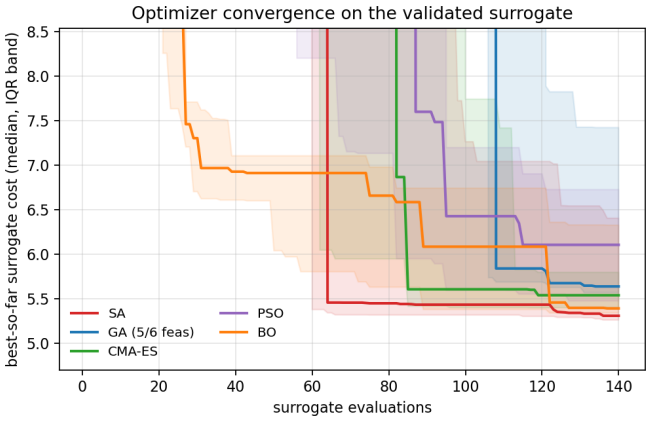

Fig. 5. Convergence of the five optimizers on the validated surrogate (median best-so-far over six seeds, IQR band). The top three converge to a statistically indistinguishable cost; random placement never enters the feasible band.

within noise (6.07 ± 0.72 vs. the comparison's simulated annealing 5.91 ± 1.03), confirming the extension is sound.

Opening the base search, however, does not reduce cost: full-cell optimization yields 5.61 ± 0.50 (five of six seeds feasible) versus 6.07 ± 0.72 fixed-base—statistically equal (Table V)—even though the optimizer does move the base substantially (mean displacement 0.42 m, yaw up to 1.17 rad). This is a structural property of the problem, not a search failure: the joint-travel surrogate and every validity constraint (reach, inter-station clearance, no-tilt,

TABLE V
Fixed-base vs. full-cell synthesis (base pose added to the decision space), six seeds.

formulation	cost (mean±std)	feasible	base displacement
fixed base	6.07 ± 0.72	6/6	0
full cell	5.61 ± 0.50	5/6	0.42 m (max 0.64)

base keep-out) are computed in the robot base frame, so the entire cell is invariant under a rigid translation and rotation. The absolute base pose therefore provides no cost gradient—for cycle-time optimization, only the station geometry relative to the robot matters. Pinning the absolute mount, for floor footprint or accessibility, would require adding a corresponding objective term that the cycle-time surrogate deliberately does not contain. A synthesized full-cell configuration with a displaced, yawed base (relocated 0.57 m, yaw 0.36 rad) nonetheless executes end-to-end in the twin (IK guard passed, warmup relay clean at 140.3 s, no planning failures), confirming the base-yaw realization is geometrically correct.

VIII. Generalization at Scale and Multi-Robot Transfer

The preceding results use a handful of seeds on a UR5. We now test whether the approach generalizes across many configurations and across robot arms. Per the base-pose finding, the base is fixed at the arena-center mount for both robots.

Scale. On each of fifty seeds, simulated annealing against the surrogate beats the equal-budget random-valid baseline: 46/50 for the UR5 (mean −9.2%) and 45/50 for a UR10 (mean −7.3%), with optimized-cost distributions of 4.99 ± 0.16 and 4.85 ± 0.26 respectively (Table VI, Fig. 6). Simulated annealing reliably beats random across fifty seeds on both arms.

Multi-robot transfer. The UR10 is introduced by a single ur_type parameter: the Universal Robots arms share joint names, so the same semantic model, planning group, gripper mount, and tool frame load unchanged, while the UR10's longer links flow automatically into the inverse

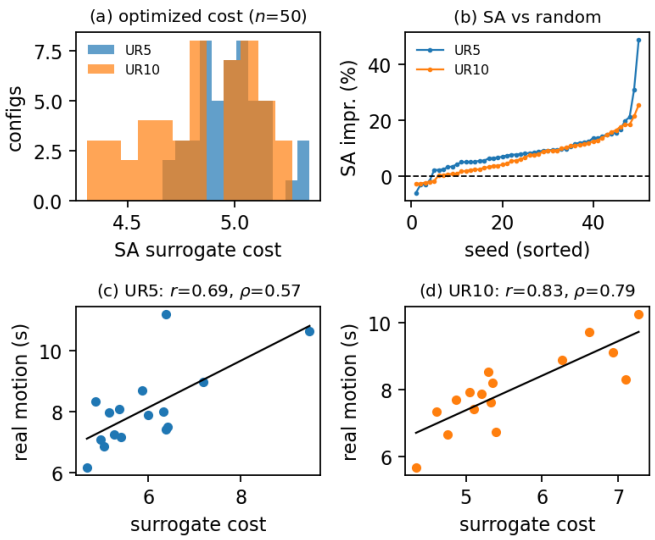

Fig. 6. Generalization across 50 seeds and two robot arms. (a) Optimized-cost distribution. (b) Per-seed improvement of simulated annealing over the equal-budget random baseline (sorted); SA wins 46/50 (UR5) and 45/50 (UR10). (c,d) Surrogate vs. real RRTstar motion at scale ($n = 16$, median estimator) for UR5 (full planning budget) and UR10 (reduced budget); fit lines shown, bootstrap statistics in Table VII.

TABLE VI
Simulated annealing vs. equal-budget random across 50 seeds, two robot arms.

robot	SA cost (mean±std)	SA beats random	mean impr.
UR5	4.99 ± 0.16	46/50	−9.2%
UR10	4.85 ± 0.26	45/50	−7.3%

kinematics and hence the surrogate. The arena, annulus, and task are identical to the UR5.

Real-motion correlation at scale. We validate sixteen configurations per robot against the real RRTstar planner, spanning the surrogate range, with bootstrap 95% confidence intervals (Table VII, Fig. 6). The surrogate correlates with real motion on both arms, both intervals excluding zero: UR10 Pearson $r = 0.828$ $[0.62, 0.95]$, Spearman $\rho = 0.791$ $[0.41, 0.96]$; UR5 $r = 0.694$ $[0.31, 0.93]$, $\rho = 0.574$ $[0.04, 0.88]$. At scale we use the median planned duration as the estimator rather than the minimum: the median is noisier because non-converged RRTstar plans contaminate it, whereas the curated gate of Section V used the minimum as the principled converged-optimal estimate. We therefore do not claim the gate's $r = 0.88$ at scale; the curated gate remains the primary validation, and these at-scale figures are an honest, lower-bounding generalization check under a noisier estimator and a much smaller per-configuration planning budget.

IX. High-Fidelity Twin Execution

To confirm that an optimized configuration is not merely fast on paper, we executed a feasible optimized winner

TABLE VII
Surrogate vs. real RRTstar motion at scale ($n = 16$ per robot, median estimator, bootstrap 95% CI). Both intervals exclude zero.

robot	Pearson r [95% CI]	Spearman ρ [95% CI]
UR10	0.828 [0.62, 0.95]	0.791 [0.41, 0.96]
UR5	0.694 [0.31, 0.93]	0.574 [0.04, 0.88]

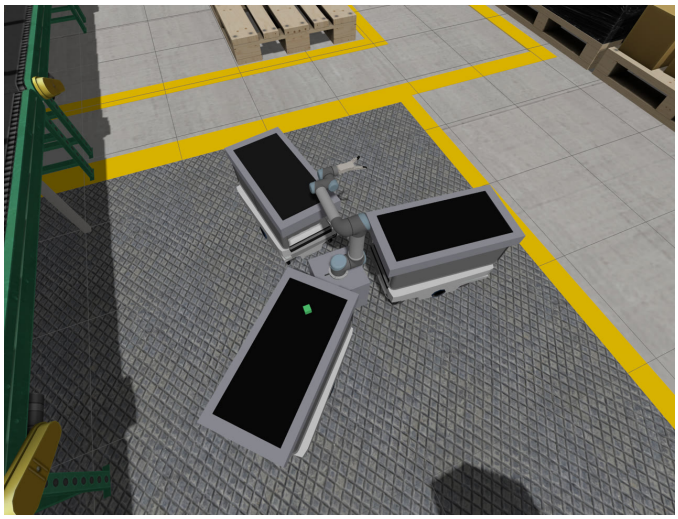

Fig. 7. Twin execution of the optimized winner val_0: the UR5 + RG2 arm reaches into a conveyor station mid-relay. The three stations are pinwheeled around the arm in the optimizer's chosen layout; the welded part (green cube) is carried between stations. Both trials complete the full pick–place–pick–place relay with no planning failures (Table VIII).

(val_0, surrogate 5.17) in the full high-fidelity twin: Gazebo Harmonic GUI, the complete move_group stack, the real RG2 gripper, and a welded-cube carry. The IK guard passed (all three stations reachable), and the four-operation relay (pick–place–pick–place) ran end-to-end across a warmup and a measured trial, both succeeding with the gripper verified clear at start and no planning failures (Table VIII, Fig. 7).

X. Discussion and Limitations

The result is bounded in an important way. The most aggressive minimal-travel winners produced by the optimizer place stations close together; while the headless RRTstar probe (with belt exclusion and joint continuity) solves these, the twin's default randomized planner (RRT-Connect with the real gripper collision model and a welded carry) cannot execute the cramped later segments—the first transfer succeeds, then a subsequent segment fails to plan. Thus our guarantees separate cleanly: the surrogate is validated as a predictor of optimizing-planner motion time, and an optimized configuration is demonstrated executable in the twin, but the single most aggressive optimum is not necessarily both. A natural extension is to fold a twin-executability constraint (e.g., a minimum inter-station clearance, or planning with the same planner the

TABLE VIII
Full-twin relay execution of optimized winner val_0.

run	cycle (s)	success	gripper clear	failure
warmup	133.0	yes	yes	none
trial 1	148.0	yes	yes	none

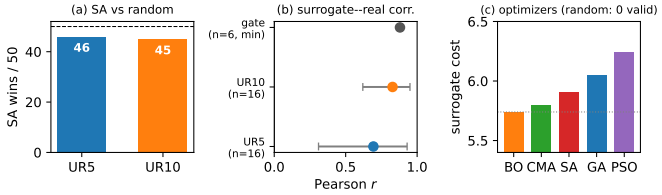

Fig. 8. The quantitative story at a glance. (a) Simulated annealing beats the equal-budget random baseline 46/50 (UR5) and 45/50 (UR10). (b) Surrogate–real correlation: the curated gate ($r = 0.88$, $n = 6$, min estimator) and the at-scale validations ($n = 16$, median estimator, bootstrap 95% CI). (c) Five optimizers on the same surrogate—the top three (BO, CMA-ES, SA) tie, while random placement yields no valid layout.

twin uses) directly into the surrogate or as an optimization constraint.

Several further points bound the at-scale results. The relatively wide confidence intervals in Table VII reflect the cost ceiling of real-sim validation: RRTstar is an optimizing planner that consumes its full planning budget on every evaluation (on the order of minutes per configuration at the gate's budget), which limits how many configurations can be validated per robot—a practical constraint, not a flaw in the surrogate or optimizer. The UR5 at-scale correlation ($r = 0.694$) is moderate compared with the curated gate ($r = 0.880$); this is explained by the estimator choice—at scale we report the noisier median duration, whereas the gate uses the converged-optimal minimum—rather than by any change in the underlying relationship. The curated validation gate (Section V) uses six configurations; a larger held-out set would tighten the gate correlation estimate and is a natural extension. The base-pose result shows that pinning the absolute mount for footprint or accessibility would require an additional objective term beyond cycle time. Finally, grasping is welded (contact dynamics are out of scope) and the relay uses three stations and a single part type; scaling the station count and part set, and transferring to real hardware, are future work—the closest current evidence for the latter being the end-to-end twin executions of Sections VII and IX.

XI. Conclusion

We presented a reconfigurable robotic pick-and-place cell whose layout is optimized against a deterministic joint-travel surrogate, and validated that surrogate against a high-fidelity twin with explicit attention to honesty: two documented negative results precede the positive one, and the boundary of the positive result is stated. When the real measurement is aligned with what the surrogate models—an optimizing planner, kinematic continuity, and a fixed visit order—the surrogate predicts converged-optimal motion time ($r = 0.88$, $\rho = 0.94$). Simulated annealing against the surrogate beats an equal-budget baseline 5/5 (-17.4%), and the gain transfers to real motion (-19.4%) and to a clean end-to-end relay in the twin. Building on this, a fair five-optimizer comparison shows that no method dominates and that simulated annealing is justified by its cost and reliability; extending the search to the robot base pose exposes a structural property—base placement is cost-neutral under a cycle-time objective, so only relative station geometry matters; and the approach generalizes across 50 seeds and two robot arms (UR5 and UR10), with the surrogate–real correlation transferring under bootstrapped confidence intervals. The methodology—validate the proxy as rigorously as the optimization, report where it stops holding, and test whether it generalizes—is the central takeaway; Fig. 8 collects the headline quantitative results.

References

[1] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2000, pp. 995–1001.

[2] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," Int. J. Robot. Res., vol. 30, no. 7, pp. 846–894, 2011.

[3] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671–680, 1983.

[4] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: a MoveIt! case study," J. Softw. Eng. Robot., vol. 5, no. 1, pp. 3–16, 2014.

[5] I. A. Şucan, M. Moll, and L. E. Kavraki, "The Open Motion Planning Library," IEEE Robot. Autom. Mag., vol. 19, no. 4, pp. 72–82, 2012.

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Sci. Robot., vol. 7, no. 66, 2022.

[7] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," Proc. IEEE, vol. 104, no. 1, pp. 148–175, 2016.

[8] N. Hansen, "The CMA evolution strategy: A tutorial," arXiv:1604.00772, 2016.

[9] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," IEEE Access, vol. 8, pp. 89497–89509, 2020.

[10] T. Kawabe, T. Nishi, Z. Liu, and T. Fujiwara, "Surrogate-assisted motion planning and layout design of robotic cellular manufacturing systems," Eng. Appl. Artif. Intell., vol. 150, art. 110530, 2025.

[11] R.-A. Sánchez-Sosa and E. Chavero-Navarrete, "Robotic cell layout optimization using a genetic algorithm," Appl. Sci., vol. 14, no. 19, art. 8605, 2024.

[12] D. Barral, "Optimization tool for assembly workcell layout," U.S. Patent 6 470 301 B1, Oct. 22, 2002.

[13] L. Wang, Z. Wang, K. Gumma, A. Turner, and S. Ratchev, "Multi-agent cooperative swarm learning for dynamic layout optimisation of reconfigurable robotic assembly cells based on digital twin," J. Intell. Manuf., vol. 36, no. 5, pp. 2959–2982, 2025.

[14] M. Kaimujjaman, T. Nishi, and T. Fujiwara, "Hierarchical optimization of robotic placement, motion planning, and posture for multi-target manipulation," Appl. Sci., vol. 15, no. 22, art. 11941, 2025.

[15] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," IEEE Trans. Robot. Autom., vol. 12, no. 4, pp. 566–580, 1996.
[16] S. M. LaValle, Planning Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2006.
[17] C. Saavedra Sueldo, I. Perez Colo, M. De Paula, S. A. Villar, and G. G. Acosta, "ROS-based architecture for fast digital twin development of smart manufacturing robotized systems," Ann. Oper. Res., 2022, doi: 10.1007/s10479-022-04759-4.
[18] N. Kousi, C. Gkournelos, S. Aivaliotis, K. Lotsaris, A. C. Bavelos, P. Baris, G. Michalos, and S. Makris, "Digital twin for designing and reconfiguring human–robot collaborative assembly lines," Appl. Sci., vol. 11, no. 10, art. 4620, 2021.
[19] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, The Traveling Salesman Problem: A Computational Study. Princeton, NJ: Princeton Univ. Press, 2006.
[20] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405–2415, 2019.
[21] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," Swarm Evol. Comput., vol. 1, no. 2, pp. 61–70, 2011.
[22] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2004, pp. 2149–2154.
[23] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proc. IEEE Int. Conf. Neural Netw. (ICNN), 1995, pp. 1942–1948.
[24] J. H. Holland, Adaptation in Natural and Artificial Systems. Cambridge, MA: MIT Press, 1992.
[25] Universal Robots, "UR5 collaborative industrial robot arm—technical specifications," https://www.universal-robots.com/products/ur5-robot/; and OnRobot, "RG2 flexible two-finger robot gripper—datasheet," https://onrobot.com/en/products/rg2-gripper (both accessed Jun. 2026).